

Syllabus

Module I: Foundations of Generative AI & LLMs (2 days)

Goal: Understand core concepts.

Topics:

1. Generative AI Essentials

- What is Generative AI?
- Generative vs discriminative AI
- LLMs vs other AI model types (Vision, Audio, Diffusion, Reinforcement Learning)
- Key use cases:
 - i. Intelligent chatbots
 - ii. Document understanding
 - iii. Coding assistants
 - iv. Automation & decision support

2. Large Language Model Fundamentals

- How LLMs work
- Tokens, tokenization, embeddings
- Context windows and memory limits
- Decoding parameters (temperature, top-p, top-k, frequency penalties)
- Roles in LLM conversations: system, user, assistant

3. Embedding & Vector Basics

- What embeddings represent
- Vector similarity & distance metrics
- How semantic search works
- Embedding models vs LLM models
- Meaning of "vector databases"

4. Prompt Engineering

- Prompting basics
- Zero-shot, one-shot, few-shot prompting
- Chain-of-Thought (CoT), Tree-of-Thought (ToT)
- Structured prompting, guardrails, and role prompts

Module II: Working with LLMs (2 days)

Goal: Goal: Gain hands-on skills to use the most common LLM tools and APIs.

Topics:

1. Using OpenAI / Anthropic / Google APIs

- Chat vs Completions APIs
- Temperature, max_tokens, roles
- Building structured outputs
- Error handling, retries, rate limits
- Cost optimization & batching

2. Working With Embeddings APIs

- Generating embeddings for documents and queries
- Choosing embedding models
- Semantic search fundamentals
- Normalization & vector arithmetic

3. Local LLMs & Edge Models

- Installing Ollama, Llama models
- Running LLMs offline / on-device
- Local embeddings
- When to choose local vs cloud models

Module III: Retrieval Augmented Generation (RAG) (2 days)

Goal: Build LLM systems grounded in external knowledge.

Concepts:

1. Document Processing & Chunking

- Chunking strategies (semantic, fixed, recursive)
- Embedding large documents effectively
- Storing vectors in FAISS / Chroma

2. Retrieval Mechanisms

- Vector similarity search
- Sparse retrieval: BM25
- Hybrid retrieval: BM25 + vectors
- Metadata filtering
- Multi-query retrieval
- Reranking (LLM-based, cross-encoder based)

3. Context Injection Strategies

- Short-term context (per-query retrieval)
- Persistent context (long-term memory)
- Structured context insertion
- Summarization-based compression
- Context window optimization

4. Hands-On Implementation

- Build vector search using FAISS / Chroma
- RAG pipeline *without* LangChain (pure Python)
- Handle token limits, overlapping chunks
- Evaluate retrieval quality

Module IV: LangChain Essentials (3 days)

Goal: Learn how to build production AI workflows using LangChain's abstractions.

1. LangChain Fundamentals:

- What is LangChain and why it exists
- LangChain Expression Language (LCEL)
- Key components: Models, Prompts, Output Parsers, Tools, Memory, Agents

2. LLM & Embedding Models in LangChain

- ChatOpenAI, ChatOllama, Bedrock models
- Embedding models (OpenAI, HuggingFace, Nomic, Ollama embeddings)
- Selecting the right model for latency, accuracy, cost

3. Prompts & Output Handling

- PromptTemplate
- ChatPromptTemplate & structured prompts
- Output Parsers (structured output, pydantic)
- Guardrails & safe outputs

4. Tools & Tool Calling:

- Creating tools with LangChain decorators
- Built-in tools
- Integrating external APIs
- Using tools in agent workflows

2. Memory Systems in LangChain

- ConversationBufferMemory
- ConversationBufferWindowMemory
- Entity memory
- Vector-store backed memory
- Short-term vs long-term memory via LangChain

3. Retrievers & Vector Stores in LangChain

- Chroma, FAISS, PGVector, Pinecone
- Building retrievers
- Compression retrievers
- Multi-vector & multi-source retrievers
- Filters and rerankers

4. RAG with LangChain

- Basic RetrievalQA
- ConversationalRetrievalQA
- LCEL-based RAG pipeline
- Advanced RAG with post-processing & reranking

5. Agents in LangChain

- Agent types (ReAct, OpenAI Functions, XML Agents, Tool Calling Agents)
- AgentExecutor
- Multi-step reasoning
- Tool-use + retrieval integration
- Limitations & debugging agents

Module V: LangGraph: Agent Workflows & Multi-Agent Systems (3 days)

Goal: Build reliable, production-ready agent workflows using LangGraph.

Topics:

- StateGraph: the core building block
- Nodes, edges, parallel branches
- Entry points & END nodes
- Checkpointing & persistent state
- Interrupts, human-in-the-loop
- Building multi-agent systems
- Building agentic RAG using graphs
- Using LangGraph with Ollama / OpenAI
- Reliability, retries, recoverability

Module VI: Agents & Structured Tool Calling (2 days)

Goal: Enable LLMs to reason, plan, and take actions.

Concepts:

1. Agentic AI

- What is an agent
- Single-agent vs multi-agent workflows
- Agentic RAG (dynamic retrieve → reason → improve)

2. LLM Tool/Function Calling

- Defining tools with schemas
- OpenAI/Anthropic function calling
- Validating and parsing structured outputs
- Safety, constraints, and reliability

3. Agent Implementation in Python

- Building tools (math, APIs, DB queries, utilities)
- Self-correcting loops
- Memory handling
- Logging and traceability

Module VII: Multi-Agent Systems (2 days)

Goal: Enable collaboration between specialized agents.

Concepts:

6. Role-specific agents:

- Planner, Retriever, Analyzer, Critic, Writer

7. Multi-agent communication patterns

8. Conflict resolution & feedback cycles

9. Workflow orchestrators:

- LangGraph
- Function pipelines
- Message-passing systems

Module VIII: Advanced & Production RAG (2 days)

Goal: Architect scalable, enterprise-grade AI systems.

Topics:

1. Multi-query retrieval optimization
2. Advanced re-ranking (cross-encoder scoring)
3. Context fusion and compression
4. Chunk scoring & prioritization
5. Caching strategies
6. Hallucination reduction techniques
7. Query planning & multi-hop reasoning
8. Cost and latency optimization

Practical Work

- Build an enterprise RAG pipeline
- Hybrid search implementation
- Inject persistent + short-term memory
- Build monitoring dashboards for retrieval quality

Module IX: Model Context Protocol (MCP) (2 days)

Goal: Learn how LLMs securely connect to tools, systems, APIs, and external environments.

1. MCP Fundamentals

- What is MCP?
- Why MCP exists
- How LLMs use MCP to interact with tools
- Difference between:
 - i. Tools
 - ii. Extensions
 - iii. Routines
 - iv. Servers

10. Built-in No-Code MCP Servers

- Filesystem
- Browser
- Fetch (API calling)
- Git
- SQLite
- JSON store (Depending on the platform)

11. Building Custom MCP Servers

- Minimal Python MCP server
- Server configuration (manifest, permissions)
- Defining custom tools
- Streaming responses
- Debugging MCP processes

12. MCP in Applications

- Using MCP with ChatGPT
- Using MCP with LangGraph / Agents
- Secure execution & sandboxing
- When to use MCP vs native APIs

13. Real Use Cases

- MCP-based RAG
- MCP-based automation assistant

- MCP as a backend for multi-agent systems
- Enterprise tool integrations (DB, cloud, internal APIs)

14. Hands-on Project

- Build your own MCP server (example: TODO manager, email sender, finance fetcher, Notion client, etc.)

Module X: Projects for Hands-on

Goal: Build real-world systems end-to-end.

Project Ideas:

- LangChain-powered RAG chatbot
- Tool-enabled LangChain agent
- LangChain + LangGraph multi-agent workflow
- AI Email assistant using function calling + LangChain
- Finance/legal domain expert using embeddings + RAG
- Local ChatGPT clone using ChatOllama + LangChain
- Multi-modal RAG (text + image + audio)
- Embedding compression & quantization
- LLM orchestration frameworks
- Graph-based agent systems
- Memory architectures (episodic, semantic, long-term)
- Custom MCP servers & agent ecosystems
- Continual learning & adaptive RAG